

INFORME PRACTICA FSO

Integrants:

Arnau Solé de la Cruz

Dario Alexander Plesa Plesa

Index

Fase1:	4
Pas 0.1:.....	4
Especificacions:	4
Implementacio:	4
Pas 0.2 Reestructuració de les funcions	5
Especificacions:	5
Implementacio:	5
Pas 1.1 Creació del programa de la Pilota (pilota1.c).....	6
Especificacions:	6
Implementacio:	6
Pas 1.2: Modificació del programa principal (mur1.c) i control del temps	7
Especificacions:	7
Implementacio:	7
Pas 1.3 : Dinàmica de col·lisions i aparició de noves pilotes (B)	9
Especificacions:	9
Implementacio:	9
Pas 1.4 : El gestor del joc (mur1.c) i el control del cronòmetre	9
Especificacions:	9
Implementacio:	10
Pas 1.5 : Independència de dades i gestió local del procés.....	10
Especificacions:	10
Implementacio:	10
Jocs de proves:.....	11
Fase2:	12
Pas 2.1: Identificació de les Seccions Crítiques	12
Especificacio:.....	12
Implementacio:	12
Pas 2.2: Identificació de les Seccions Crítiques	13
Especificacio:.....	13
Implementacio:	13
Pas 2.3: Identificació de les Seccions Crítiques	13
Especificacio:.....	13
Implementacio:	13
Pas 2.4: Identificació de les Seccions Crítiques	14
Especificacio:.....	14

Implementacio:	14
Pas 2.5: Identificació de les Seccions Crítiques	15
Especificacio:.....	15
Implementacio:	15
Jocs de proves:.....	15
Fase3:	16
Pas 3.1: Configuració Dinàmica i Mida Variable	16
Especificacio:.....	16
Implementacio:	16
Pas 3.2: Configuració Dinàmica i Mida Variable	16
Especificacio:.....	16
Implementacio:	16
Pas 3.3: Configuració Dinàmica i Mida Variable	20
Especificacio:.....	20
Implementacio:	20
Jocs de proves:.....	20
Fase4:	21
Pas 4.1: Blocs especials i temporitzador en memòria compartida	21
Especificacio:.....	21
Implementacio:	21
Pas 4.2: Activació de super-poders i destrucció de murs.....	21
Especificació:.....	21
Implementació:	21
Pas 4.3: Rol del pare en el control del temporitzador.....	22
Especificació:.....	22
Implementació:	22
Pas 4.4: Comunicació de dades entre pilots (Bústia).....	23
Especificació:.....	23
Implementació:	23
Pas 4.5: Sincronització entre els fils d'un mateix procés.....	24
Especificació:.....	24
Implementació:	24
Jocs de proves:.....	26

Fase1:

Pas 0.1:

Especificacions:

Abans de realitzar la separació, cal entendre com es distribueixen les dades. En utilitzar winsuport2, el tauler de joc ja no resideix en una variable local simple, sinó en un bloc de memòria al qual ambdós processos hauran de tenir accés.

Per fer-ho, dividirem el codi en dos executables separats: mur1 (el gestor del joc i la paleta) i pilota1

(el programa que controla el moviment de cada pilota).

```
$ cp mur0.c mur1.c
```

Implementacio:

```
/* --- Variables Globals --- */
/* Variables de l'entorn de joc */
int n_fil, n_col;      /* dimensions del camp de joc */
int m_por;             /* mida de la porteria (en caracters) */
int retard;            /* valor del retard de moviment, en mil.lisegons */
char strin[LONGMISS]; /* variable per a generar missatges de text a la
pantalla */

/* Variables de la paleta */
int f_pal, c_pal; /* posicio del primer caracter de la paleta (fila,
columna) */
int m_pal;        /* mida de la paleta (en caracters) */
int dirPaleta = 0; /* direcció de moviment de la paleta */

/* Variables de la pilota */
int f_pil, c_pil; /* posicio de la pilota, en valor enter (per pintar a
pantalla) */
float pos_f, pos_c; /* posicio real de la pilota, en valor real (per a
moviments suaus) */
float vel_f, vel_c; /* velocitat de la pilota (components horitzontal i
vertical) */

/* Variables globals per a la memòria compartida (IPC) */
int id_mem; /* identificador de la memòria compartida creada */
void *p_mem; /* punter cap a la zona de memòria mapejada */

int minuts, segons;
int comptador_retard = 0;
```

Pas 0.2 Reestructuració de les funcions

Especificacions:

El programa s'organitza ara de manera que les responsabilitats quedin dividides: `int carrega_configuracio(FILE *fit);` Segueix al pare per llegir els paràmetres inicials. `int inicialitza_joc(void);` El pare reserva la memòria compartida i dibuixa el mur inicial. `int mou_pilota(void);` Aquesta funció es trasllada a `pilota1.c`, on s'executarà en un bucle infinit dins del procés fill. També es traslladaran totes les funcions a les que fa referència el moviment de la pilota. `int mou_paleta(void);` Es manté al procés pare (`mur1.c`) per permetre al jugador controlar el joc. Pràctica 2 FSO.

Implementacio:

main de `mur1.c` on es pot veure la implementacio de `carrega_configuracio`, `inicialitza_joc` i `mou_paleta`

```
/* --- Programa Principal --- */
int main(int n_args, char *ll_args[])
{
    /* 2. Càrrega del fitxer i configuració del retard del joc */
    if (carrega_configuracio(fit_conf) != 0)
        exit(3);

    /* 3. Inicialització de la memòria compartida i del taulell gràfic */
    if (inicialitza_joc() != (pid_t)0)
        exit(4);

    char s_id_mem[10], s_fil[10], s_col[10], s_retard[10];
    char s_pos_f[10], s_pos_c[10], s_vel_f[10], s_vel_c[10];
    char s_c_pal[10], s_m_pal[10];

    /* 4. Bucle principal d'execució del joc */
    do
    {
        fi1 = mou_paleta(); /* Moure la paleta i llegir teclat */
    }
```

main de `pilota1.c` on es pot veure la implementacio de `mou_pilota`

```
int main(int n_args, char *ll_args[])
{
    do
    {
        fi2 = mou_pilota();
        win_retard(retard);
    } while (!fi2);
}
```

Pas 1.1 Creació del programa de la Pilota (pilota1.c)

Especificacions:

Per tal que la pilota es mogui de forma concurrent, crearem un nou fitxer codi font anomenat pilota1.c que compilarà com a un executable separat. Tasques: Aquest programa necessitarà un main() propi. Com que la memòria compartida ja està creada pel procés principal, pilota1 ha de rebre per arguments de la línia de comandes (via argv) la informació necessària: l'identificador de la memòria compartida (id_mem), les dimensions del tauler, la posició inicial de la pilota, la velocitat i el retard. Un cop convertit l'id_mem a enter, el procés ha de cridar map_mem(id_mem) per obtenir el punter, i connectar-se al tauler amb win_set(punter, files, columnes). Pràctica 2 FSO curs 2025-2026 DEIM (ETSE) - URV 8 Moveu la funció mou_pilota() a aquest fitxer. Ara el bucle principal de pilota1 només consistirà a moure la pilota i fer un retard (win_retard()). Atenció: el procés pilota no ha de cridar win_update() ni llegir el teclat!

Implementacio:

```
int main(int n_args, char *ll_args[])
{
    if (n_args < 12)
        exit(1);

    id_mem = atoi(ll_args[1]);
    n_fil = atoi(ll_args[2]);
    n_col = atoi(ll_args[3]);
    pos_f = atof(ll_args[4]);
    pos_c = atof(ll_args[5]);
    vel_f = atof(ll_args[6]);
    vel_c = atof(ll_args[7]);
    retard = atoi(ll_args[8]);
    c_pal = atoi(ll_args[9]);
    m_pal = atoi(ll_args[10]);
    numero = (char)ll_args[11][0];
    num_pilota = atoi(ll_args[11]);

    comp = map_mem(id_mem);
    win_set(&(comp->tauler), n_fil, n_col);
    comp->npilotes = comp->npilotes + 1;

    do
    {
        fi2 = mou_pilota();
        win_retard(retard);
    } while (!fi2);

    return 0;
}
```

Pas 1.2: Modificació del programa principal (mur1.c) i control del temps

Especificacions:

Ara el programa principal assumirà el rol de gestor del joc i de la paleta. Tasques a la funció main: Creació del procés pilota: Després d'inicialitzar la memòria compartida i el tauler, feu un fork() i un exec() (per exemple, execlp) per posar en marxa l'executable ./pilota1. Passeu-li tots els arguments necessaris com a text (usant sprintf). Control del temps i bucle principal: El bucle d'aquest procés pare ara només ha d'invocar la funció de moviment de la paleta i win_update(). A més, haurà de controlar el temps de joc (minuts:segons) i mostrar-lo contínuament per la línia de missatges (usant win_escristr()), fins que es doni alguna condició de finalització. Missatges de finalització: L'última acció del programa, just abans de tancar, serà la d'escriure el temps total de joc per la sortida estàndard (amb un printf), juntament amb els missatges de finalització proposats en la versió d'exemple original. Proveu els canvis generant els executables (per exemple \$ make mur1). ATENCIÓ: podria ser que la pilota es mogués molt ràpid i no la vegéssiu. Assegureu-vos d'ajustar el paràmetre de retard al llançar el programa.

Implementacio:

- Creacio del proces pilota:

```
/* 3. Inicialització de la memòria compartida i del taulell gràfic */
if (inicialitza_joc() != (pid_t)0)
    exit(4);

char s_id_mem[10], s_fil[10], s_col[10], s_retard[10];
char s_pos_f[10], s_pos_c[10], s_vel_f[10], s_vel_c[10];
char s_c_pal[10], s_m_pal[10];

/* Conversió de dades a string */
sprintf(s_id_mem, "%d", id_mem);
sprintf(s_fil, "%d", n_fil);
sprintf(s_col, "%d", n_col);
sprintf(s_retard, "%d", retard);
sprintf(s_pos_f, "%.2f", pos_f);
sprintf(s_pos_c, "%.2f", pos_c);
sprintf(s_vel_f, "%.2f", vel_f);
sprintf(s_vel_c, "%.2f", vel_c);
sprintf(s_c_pal, "%d", c_pal);
sprintf(s_m_pal, "%d", m_pal);

if (fork() == 0)
{
```

```

        /* Passem els arguments com a cadenes de text */
        execlp("./pilota1", "pilota1", s_id_mem, s_fil, s_col,
                s_pos_f, s_pos_c, s_vel_f, s_vel_c, s_retard, s_c_pal,
s_m_pal, (char *)"1", (char *)NULL);
        exit(1);
    }

```

- Control del temps i bucle principal:

```

/* 4. Bucle principal d'execució del joc */
do
{
    fi1 = mou_paleta(); /* Moure la paleta i llegir teclat */
    comptador_retard += retard;
    if (comptador_retard >= 1000) /* Ha passat 1 segon */
    {
        segons++;
        if (segons == 60)
        {
            minuts++;
            segons = 0;
        }
        comptador_retard = 0;
    }
    sprintf(strin, "Temps: %02d:%02d | Blocs: %d | Pilotes: %d",
minuts, segons, comp->nblocs, comp->npilotes);
    win_escriptr(strin);

    win_update(); /* Bolcar els canvis fets a la memòria a la
pantalla física */
    win_retard(retard); /* Pausar el procés el temps establert abans
del següent frame */
} while (!fi1 && comp->nblocs > 0 && comp->npilotes > 0); /* Sortir
si demanem sortir (!fi1) o acaba la partida (!fi2) */

```

- Missatges de finalització:

```

/* 5. Comprovació de final de joc i missatges de sortida */
if (comp->nblocs == 0)
    mostra_final("YOU WIN !");
else
    mostra_final("GAME OVER");

win_fi();

printf("Temps de joc -> %02d:%02d\n", minuts, segons);

```


Pas 1.3 : Dinàmica de col·lisions i aparició de noves pilotes (B)

Especificacions:

Un cop la primera pilota es mou correctament, el següent objectiu és gestionar la destrucció de blocs. Cal modificar la lògica de col·lisions a pilota1.c per detectar l'impacte amb blocs de tipus 'B'. Quan això passi: Creació concurrent: El propi procés pilota invocarà un fork() i un execlp() per instanciar un nou executable ./pilota1. Estat inicial: La nova pilota s'ha de crear en la mateixa posició del bloc destruït, però invertint la seva velocitat perquè realitzi l'efecte de rebot immediat. Nota sobre el rendiment visual: En aquesta fase, com que encara no utilitzem semàfors, és totalment normal observar "problemes en l'entorn gràfic": caràcters que desapareixen, parets que s'esborren parcialment o pilotes que "mengen" trossos de la paleta. Aquests són els conflictes clàssics de la memòria compartida sense sincronització.

Implementació:

```
        if (fork() == 0)
        {
            /* Passem els arguments com a cadenes de text */
            execlp("./pilota1", "pilota1", s_id_mem, s_fil,
s_col,
                    s_pos_f, s_pos_c, s_vel_f, s_vel_c,
s_retard, s_c_pal, s_m_pal, s_numero, (char *)NULL);
            exit(1);
        }
    }
```

Pas 1.4 : El gestor del joc (mur1.c) i el control del cronòmetre

Especificacions:

En aquesta fase, el programa principal es converteix en el mestre de cerimònies. Ja no mou la pilota directament, sinó que gestiona l'entorn: Instanciació inicial: Abans d'entrar al bucle, el pare fa el primer fork() per llançar la pilota original, passant-li l'identificador de la memòria compartida (id_mem) i els paràmetres inicials via argv. Bucle de refresc: El bucle del pare ara és més lleuger. Només ha d'executar mou_paleta() per recollir el input de l'usuari i, molt important, cridar a win_update() per bolcar els canvis de la memòria a la pantalla física. Control del temps: Cal implementar un cronòmetre en format minuts:segons. Com que coneixem el valor de retard, podem usar un comptador intern, no s'han d'usar les rutines de la llibreria Time.h. Aquest temps s'ha d'imprimir a la part inferior del tauler amb win_escriure(). Finalització: Quan es compleixi una condició de final (no queden blocs, no queden pilotes o es prem RETURN), el programa tancarà les curses i imprimirà per la sortida estàndard (printf) el temps total de la partida i el resultat del joc.

Implementacio:

```
/* 4. Bucle principal d'execució del joc */
do
{
    fi1 = mou_paleta(); /* Moure la paleta i llegir teclat */
    comptador_retard += retard;
    if (comptador_retard >= 1000) /* Ha passat 1 segon */
    {
        segons++;
        if (segons == 60)
        {
            minuts++;
            segons = 0;
        }
        comptador_retard = 0;
    }
    sprintf(strin, "Temps: %02d:%02d | Blocs: %d | Pilotes: %d",
minuts, segons, comp->nblocs, comp->npilotes);
    win_escriustr(strin);

    win_update(); /* Bolcar els canvis fets a la memòria a la
pantalla física */
    win_retard(retard); /* Pausar el procés el temps establert abans
del següent frame */
} while (!fi1 && comp->nblocs > 0 && comp->npilotes > 0); /* Sortir
si demanem sortir (!fi1) o acaba la partida (!fi2) */
```

Pas 1.5: Independència de dades i gestió local del procés

Especificacions:

És vital que cada procés pilota gestioni la seva pròpia "personalitat". Tot i que el tauler de joc és un recurs compartit a la memòria IPC, dades com la posició exacta (float) i la velocitat actual han de ser variables locals dins de cada procés fill. Identificació: En crear una nova pilota des de pilota1.c, li passarem un caràcter identificador (per exemple, 'a', 'b', etc.) per poder-les distingir visualment. Localitat vs. Globalitat: Si féssim servir la memòria compartida per a la velocitat de totes les pilotes, totes es mourien alhora cap a la mateixa direcció. La gràcia dels processos independents és que cadascú té les seves pròpies dades privades, garantint un moviment totalment autònom. Consell per al laboratori: Si en fer make mur1 veieu que la pilota va "a la velocitat de la llum", reviseu que el procés fill estigui executant el seu propi win_retard() dins del bucle de pilota1.c.

Implementacio:

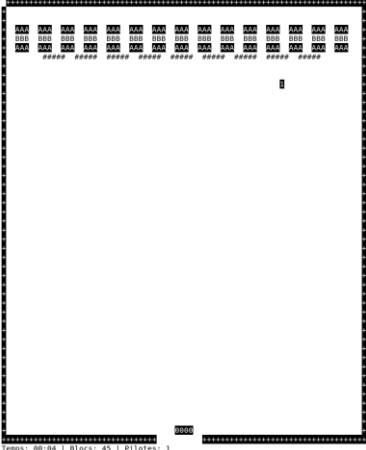
```
if (fork() == 0)
{
```

```

    /* Passem els arguments com a cadenes de text */
    execlp("./pilota1", "pilota1", s_id_mem, s_fil, s_col,
           s_pos_f, s_pos_c, s_vel_f, s_vel_c, s_retard, s_c_pal,
s_m_pal, (char *)"1", (char *)NULL);
    exit(1);
}

```

Jocs de proves:

Prova realitzada	Resultat esperat	Resultat obtingut	Conclusio
Make fase1	mur1, pilota1 compilats	<pre> milax@casa:~/Documents/GitHub/PracFS0/mur\$ make fase1 gcc -Wall -g -c mur1.c -o mur1.o gcc -Wall -g -c winsuport2.c -o winsuport2.o gcc -Wall -g -c memoria.c -o memoria.o gcc -Wall -g mur1.o winsuport2.o memoria.o -o mur1 -lcurses gcc -Wall -g -c pilotat.c -o pilotat.o gcc -Wall -g pilotat.o winsuport2.o memoria.o -o pilotat -lcurses </pre>	OK
./mur1 params_auto.txt 50	Joc executat a la mida del terminal		OK
./mur1 params_gran.txt 50	Joc executat amb els paràmetres grans		OK
./mur1 params_auto.txt 30	Joc executat amb un retard reduït	El joc va molt mes rapid	OK

Fase2:

Pas 2.1: Identificació de les Seccions Críiques

Especificacio:

Abans de programar, cal identificar els recursos compartits que requereixen accés exclusiu per evitar condicions de carrera: • El tauler (Memòria compartida): La seqüència de consultar una casella (win_quincar) i actualitzar-la (win_escricar) ha de ser atòmica; cap procés ha d'intervenir entremig. • Variables globals compartides: Qualsevol comptador situat a la memòria compartida (com el nombre de blocs o pilotes) que sigui llegit i modificat per múltiples processos ha de ser protegit. • Rutines no reentrants: Aquelles funcions que utilitzen variables globals internes (totes les rutines de les curses, excepte win_retard) si es criden de manera concurrent han de garantir l'exclusió mútua

Implementacio:

- Comprovar rebot vertical:

```
if (f_h != f_pil)
{
    waitS(id_sem);
    rv = win_quincar(f_h, c_pil);
    signalS(id_sem);
}
```

- Comprovar rebot horitzontal:

```
if (c_h != c_pil)
{
    waitS(id_sem);
    rh = win_quincar(f_pil, c_h);
    signalS(id_sem);
}
```

- Comprovar rebot diagonal:

```
if ((f_h != f_pil) && (c_h != c_pil))
{
    waitS(id_sem);
    rd = win_quincar(f_h, c_h);
    signalS(id_sem);
}
```

- Espai lliure:

```
waitS(id_sem);
if (win_quincar(f_h, c_h) == ' ')
{
    win_escricar(f_pil, c_pil, ' ', NO_INV);
}
```

Pas 2.2: Identificació de les Seccions Críiques

Especificacio:

Es farà servir un semàfor inicialitzat a 1 per actuar com a secció crítica. 1. Inicialització: Al programa principal (mur2.c), s'ha d'invocar `ini_sem(1)` abans de crear cap procés fill per obtenir l'identificador `id_sem`. 2. Comunicació d'IDs: L'identificador `id_sem` s'ha de passar com a argument de text a tots els processos pilota mitjançant `execlp()`, tal com es va fer amb la memòria compartida.

Implementacio:

```
/* 3. Inicialització de la memòria compartida i del taulell gràfic */
if (inicialitza_joc() != (pid_t)0)
    exit(4);

id_sem = ini_sem(1);
id_mis = ini_mis();
```

Pas 2.3: Identificació de les Seccions Críiques

Especificacio:

Un cop els processos coneixen `id_sem`, s'aplica el protocol d'exclusió mútua: 1. Entrada a Secció Crítica: Abans de calcular rebots o modificar el tauler, cada procés ha d'executar `waitS(id_sem)`. Si el recurs està ocupat, el procés es quedarà bloquejat fins que s'alliberi el recurs. 2. Sortida de Secció Crítica: Immediatament després d'actualitzar la memòria, s'ha d'executar `signalS(id_sem)` per permetre l'accés a altres processos. 3. Les seccions crítiques s'han de fer el més petit possible, per tal d'executar els diferents processos amb la màxima concurrència possible

Implementacio:

- Pare en `mou_paleta`:

```
if (tecla != 0)
{
    waitS(id_sem);
    if ((tecla == TEC_DRETA) && ((c_pal + m_pal) < n_col - 1))
    {
```

- Implementacio del main en pilota vist en el pas 2.1

Pas 2.4: Identificació de les Seccions Crítiques

Especificacio:

Per evitar la creació de processos orfes o problemes de jerarquia, el pare (mur2.c) serà l'únic encarregat de fer fork(). S'ha de modificar el comportament de la fase anterior per obtenir aquest comportament. 1. Cua de Missatges: El pare inicialitza la cua amb ini_mis() i passa l'identificador id_mis a les pilotes. 2. Petició de creació: Quan una pilota trenca un bloc 'B', envia un missatge asíncron al pare amb sendM() contenint les coordenades i velocitats de la nova pilota. Pràctica 2 FSO curs 2025-2026 DEIM (ETSE) - URV 11 3. Lectura no bloquejant: Atès que receiveM() és bloquejant per defecte, s'utilitzarà una estratègia per evitar que el pare es congeli. Una primera solució és que el pare s'envii un missatge de control a si mateix: si el primer missatge que llegeix és el seu, vol dir que la cua no tenia peticions de pilotes noves, s'han de llegir els missatges fins arribar al missatge que s'ha autoenviat el pare. Totes les operacions sobre la bústia són atòmiques, no cal seccions crítiques per usar-les.

Implementacio:

- Main pare

```
/* 3. Inicialització de la memòria compartida i del taulell gràfic */
if (inicialitza_joc() != (pid_t)0)
    exit(4);

id_sem = ini_sem(1);
id_mis = ini_mis();
```

- Funcio enviar_missatges en pilota:

```
void enviar_missatge(float nova_vel_f, float nova_vel_c)
{
    sprintf(missatge.s_id_mem, "%d", id_mem);
    sprintf(missatge.s_fil, "%d", n_fil);
    sprintf(missatge.s_col, "%d", n_col);
    sprintf(missatge.s_retard, "%d", retard);
    sprintf(missatge.s_pos_f, "%.2f", pos_f);
    sprintf(missatge.s_pos_c, "%.2f", pos_c);
    sprintf(missatge.s_vel_f, "%.2f", nova_vel_f);
    sprintf(missatge.s_vel_c, "%.2f", nova_vel_c);
    sprintf(missatge.s_c_pal, "%d", c_pal);
    sprintf(missatge.s_m_pal, "%d", m_pal);
    sprintf(missatge.s_id_sem, "%d", id_sem);
    sprintf(missatge.s_id_mis, "%d", id_mis);

    missatge.origen = 'F'; // F = Fill
    missatge.desti = 'P'; // P = Pare

    sendM(id_mis, &missatge, sizeof(missatge));
```

```
}
```

Pas 2.5: Identificació de les Seccions Crítiques

Especificacio:

Els recursos IPC no s'alliberen automàticament en finalitzar els processos. 1. Neteja programada: En acabar el joc, el procés principal ha d'executar obligatòriament `elim_mem()`, `elim_sem()` i `elim_mis()`. 2. Gestió d'errors: Si el programa finalitza bruscament, cal netejar manualment el sistema amb l'script `./mataipc` o les comandes `ipcs` (per llistar) i `ipcrm` (per eliminar) des del terminal.

Implementacio:

```
/* 6. Alliberament obligatori de la memòria compartida creada a l'inici */
elim_mem(id_mem);
elim_sem(id_sem);
elim_mis(id_mis);
```

Jocs de proves:

Prova realitzada	Resultat esperat	Resultat obtingut	Conclusio
Make fase2	mur2, pilota2, winsuport2, semàfor i missatge, memoria compilats	<pre>milan@casa:~/Documents/GitHub/PracFS0/mur\$ make fase2 gcc -Wall -g -c mur2.c -o mur2.o gcc -Wall -g -c winsuport2.c -o winsuport2.o gcc -Wall -g -c memoria.c -o memoria.o gcc -Wall -g -c semafor.c -o semafor.o gcc -Wall -g -c missatge.c -o missatge.o gcc -Wall -g -c mur2.o winsuport2.o memoria.o semafor.o missatge.o gcc -Wall -g -c pilota2.c -o pilota2.o gcc -Wall -g -c pilota2.o winsuport2.o memoria.o semafor.o missatge.o</pre>	OK
<code>./mur2</code> params_auto.txt 50	A l'hora de colisionar amb un bloc B, s'haurien d'evitar problemes d'asincronisme	Creacio de pilotes molt mes consistent amb les millores amb el sincronisme	OK
<code>./mur2</code> params_gran.txt 10	Reduim el retard per ampliar la possibilitat d'asincronisme	El sincronisme funciona correctament i la creació de pilotes es correcta	OK

Fase3:

Pas 3.1: Configuració Dinàmica i Mida Variable

Especificacio:

A diferència de les fases anteriors, el nombre de paletes no serà fix. S'ha de modificar la funció de càrrega per detectar línies que comencin pel caràcter 'P' al fitxer de paràmetres.

- Dades per línia: Cal llegir la fila, la columna inicial, la direcció lateral i la mida (amplada) de la paleta.

Implementacio:

```
typedef struct
{
    int f_pal, c_pal;
    int m_pal;
    int dirPaleta;
} paleta_t;
```

Pas 3.2: Configuració Dinàmica i Mida Variable

Especificacio:

La llibreria pthread requereix funcions amb la signatura void * funcio(void * arg). Cal incloure #include i afegir l'opció -lpthread al Makefile. • Moviment Lateral: Cada fil mourà la seva paleta horitzontalment de forma constant segons la velocitat rebuda pel fitxer de configuració, rebotant horitzontalment en trobar obstacles o els límits del tauler. • Mecànica del moviment vertical: Quan el fil detecti que l'usuari ha premut la tecla corresponent al seu numero, es mourà una posició cap a dalt o cap a baix. si al intentar moure's xoca amb algun element, canviarà el sentit de desplaçament vertical. • Gestió de Mida: Totes les operacions d'esborrat, dibuix i col·lisió han de gestionar el bloc complet de caràcters segons la mida definida per a cada paleta.

Implementacio:

- Implementacio mou_paleta en mur3.c

```
void * mou_paleta(void * arg){

    int num_paleta = (int)(long) arg;
    // 0 = Paleta de l'usuari
    if (num_paleta == 0){

        do{
            if (tecla_global != 0)
```



```

        {
            if ((tecla_global == TEC_DRETA) && ((c_pal + m_pal) <
n_col - 1))
            {
                /* Esborrar l'extrem esquerre i pintar el nou extrem
dret */

                waitS(id_sem);
                win_escricar(f_pal, c_pal, ' ', NO_INV);
                signalS(id_sem);
                c_pal++;
                waitS(id_sem);
                win_escricar(f_pal, c_pal + m_pal - 1, '0', INVERS);
                signalS(id_sem);
            }
            if ((tecla_global == TEC_ESQUER) && (c_pal > 1))
            {
                /* Esborrar l'extrem dret i pintar el nou extrem
esquerre */

                waitS(id_sem);
                win_escricar(f_pal, c_pal + m_pal - 1, ' ', NO_INV);
                signalS(id_sem);
                c_pal--;
                waitS(id_sem);
                win_escricar(f_pal, c_pal, '0', INVERS);
                signalS(id_sem);
            }
            if (tecla_global == TEC_RETURN){
                waitS(id_sem);
                comp->fil = 1; /* L'usuari vol sortir */
                signalS(id_sem);
            }
            dirPaleta = tecla_global;

        }
        win_retard(retard);
    } while (!comp->fil && comp->nblocs > 0 && comp->npilotes > 0);

}
else{
    int possible = -1; // Per defecte cap a dalt (-1 disminueix la
fila)
    int pal = num_paleta - 1; // Com paletes[] està en base
0 i dins de paletes no està la paleta de l'usuari, restem 1 per accedir
correctament
    do{
        if (tecla_global == (num_paleta + '0')){

            waitS(id_sem);

```

```

        for (int i = 0; i < paletes[pal].m_pal; i++){
            if (possible == 1 && win_quincar(paletes[pal].f_pal +
1, paletes[pal].c_pal + i) != ' '){
                possible = -1; // Si xoca a baix, cap a dalt
            }
            else if (possible == -1 &&
win_quincar(paletes[pal].f_pal - 1, paletes[pal].c_pal + i) != ' '){
                possible = 1; // Si xoca a dalt, cap a baix
            }
        }
        signalS(id_sem);

        if (possible != 0){
            for (int i = 0; i < paletes[pal].m_pal; i++){
                waitS(id_sem);
                win_escricar(paletes[pal].f_pal,
paletes[pal].c_pal + i, ' ', NO_INV);
                signalS(id_sem);
            }

            for (int i = 0; i < paletes[pal].m_pal; i++){
                waitS(id_sem);
                win_escricar(paletes[pal].f_pal + possible,
paletes[pal].c_pal + i, (char)(num_paleta + '0'), INVERS);
                signalS(id_sem);
            }
            paletes[pal].f_pal += possible;
        }
    }

    if (paletes[pal].dirPaleta == 1) // Dreta
    {
        if ((paletes[pal].c_pal + paletes[pal].m_pal) < n_col -
1)
        {
            /* Esborrar l'extrem esquerre i pintar el nou extrem
dret */
            waitS(id_sem);
            win_escricar(paletes[pal].f_pal, paletes[pal].c_pal,
' ', NO_INV);
            signalS(id_sem);
            paletes[pal].c_pal++;
            waitS(id_sem);
            win_escricar(paletes[pal].f_pal, paletes[pal].c_pal +
paletes[pal].m_pal - 1, (char)(num_paleta + '0'), INVERS);
            signalS(id_sem);
        }
        else
        {

```

```

        paletes[pal].dirPaleta = -1; // Canvia de direcció
    }
}
else if (paletes[pal].dirPaleta == -1) // Esquerra
{
    if (paletes[pal].c_pal > 1)
    {
        /* Esborrar l'extrem dret i pintar el nou extrem
esquerre */
        waitS(id_sem);
        win_escricar(paletes[pal].f_pal, paletes[pal].c_pal +
paletes[pal].m_pal - 1, ' ', NO_INV);
        signalS(id_sem);
        paletes[pal].c_pal--;
        waitS(id_sem);
        win_escricar(paletes[pal].f_pal, paletes[pal].c_pal,
(char)(num_paleta + '0'), INVERS);
        signalS(id_sem);
    }
    else
    {
        paletes[pal].dirPaleta = 1; // Canvia de direcció
    }
}

    win_retard(retard);
} while (!comp->fi1 && comp->nblocs > 0 && comp->npilotes > 0);

}
return NULL;
}

```

- Implementació en el main de mur3

```

// 0 = Paleta de l'usuari
pthread_create(&tid[0], NULL, mou_paleta, (void*)(long)0);

// Paletes controlades per la IA
for (int i = 1; i < npaletes; i++){
    pthread_create(&tid[i], NULL, mou_paleta, (void*)(long)i);
}

```

Pas 3.3: Configuració Dinàmica i Mida Variable

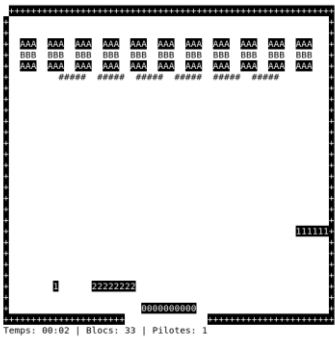
Especificacio:

Aquest és el punt crític: les paletes autònomes (fils) interaccionen amb el mateix tauler de memòria compartida que les pilotes (processos). • Semàfor compartit: Dins del fil, abans de qualsevol crida a win_quincar o win_escricar, s'ha d'utilitzar exactament el mateix semàfor IPC que fan servir els processos. • Exclusió mútua: Les crides a waitS(id_sem) i signalS(id_sem) garanteixen que cap fil o procés modifiqui el tauler mentre un altre l'està llegint o actualitzant.

Implementacio:

```
waitS(id_sem);
    for (int i = 0; i < paletes[pal].m_pal; i++){
        if (possible == 1 && win_quincar(paletes[pal].f_pal +
1, paletes[pal].c_pal + i) != ' '){
            possible = -1; // Si xoca a baix, cap a dalt
        }
        else if (possible == -1 &&
win_quincar(paletes[pal].f_pal - 1, paletes[pal].c_pal + i) != ' '){
            possible = 1; // Si xoca a dalt, cap a baix
        }
    }
    signalS(id_sem);
```

Jocs de proves:

Prova realitzada	Resultat esperat	Resultat obtingut	Conclusio
make fase3	mur3, pilota3, winsuport2, memori, semàfor, missatge compilats	<pre>* milan@casa:~/Documents/GitHub/PracF50/mur3\$ make fase3 gcc -Wall -g -c mur3.c -o mur3.o -lpthread gcc -Wall -g -c winsuport2.c -o winsuport2.o gcc -Wall -g -c memoria.c -o memoria.o gcc -Wall -g -c semafor.c -o semafor.o gcc -Wall -g -c missatge.c -o missatge.o gcc -Wall -g mur3.o winsuport2.o memoria.o semafor.o missatge.o gcc -Wall -g -c pilota3.c -o pilota3.o gcc -Wall -g pilota3.o winsuport2.o memoria.o semafor.o missatge.o</pre>	OK
./mur3 params_paleta.t xt 50	Hem de veure varies paletes i us de varios threads	 Temps: 00:02 Blocs: 33 Pilotes: 1	OK
./mur3 params_paleta.t xt 50	Les paletes han de ser capaces de pujar i baixar verticalment	Quan pulsem el numero d'una paleta, comença a pujar una casella fins que arriba de tot i despres comença a baixar	OK

Fase4:

Pas 4.1: Blocs especials i temporitzador en memòria compartida

Especificacio:

Es modifica el tauler substituint els blocs destructibles ('A') per un nou tipus de bloc especial ('T') que atorga poders temporals a les pilotes durant un període de temps pre-establert (temps_poder).

Implementacio:

```
if (temps_poder > 0) {
    temps_poder -= retard;
    if (temps_poder <= 0) {
        temps_poder = 0;
        waitS(id_sem);
        comp->poder_actiu = 0;
        signalS(id_sem);
    }
}
```

Pas 4.2: Activació de super-poders i destrucció de murs

Especificació:

Quan una pilota impacta contra un bloc 'T', el temporitzador s'incrementarà en 5 segons (acumulatiu si el poder ja estava actiu). Mentre el temps_poder sigui superior a 0: •
Efecte visual: Les pilotes es dibuixen en mode INVERS per indicar l'estat especial. •
Capacitat de destrucció: Les pilotes guanyen la capacitat de destruir parcialment els murs irrompibles (### o WLLCHAR) en impactar-hi, a més de realitzar el rebot habitual.

Implementació:

```
/* Generació dels blocs a destruir (files 3, 4 i 5) */
nb = 0;
comp->nblocs = n_col / (BLKSIZE + BLKGAP) - 1;
offset = (n_col - comp->nblocs * (BLKSIZE + BLKGAP) + BLKGAP) / 2;
for (i = 0; i < comp->nblocs; i++)
{
    for (c = 0; c < BLKSIZE; c++)
    {
        win_escriure(3, offset + c, FRNTCHAR, INVERS);
        nb++;
        win_escriure(4, offset + c, BLKCHAR, NO_INV);
        nb++;
    }
}
```

```

        win_escricar(5, offset + c, FRNTCHAR, INVERS);
        nb++;
    }
    offset += BLKSIZE + BLKGAP;
}
comp->nblocs = nb / BLKSIZE;

/* Generació de les defenses indestructibles (fila 6) */
nb = n_col / (BLKSIZE + 2 * BLKGAP) - 2;
offset = (n_col - nb * (BLKSIZE + 2 * BLKGAP) + BLKGAP) / 2;
for (i = 0; i < nb; i++)
{
    for (c = 0; c < BLKSIZE + BLKGAP; c++)
    {
        win_escricar(6, offset + c, WLLCHAR, NO_INV);
    }
    offset += BLKSIZE + 2 * BLKGAP;
}

```

Pas 4.3: Rol del pare en el control del temporitzador

Especificació:

El procés principal (mur4.c) assumeix la responsabilitat de gestionar el compte enrere dels poders. • Increment o Decrement: Dins del seu bucle, el pare decrementa la variable temps_poder segon a segon fins a arribar a 0. El pare serà l'únic que ha de veure el valor del temporitzador. Les pilotes seran les que avisaran de l'increment i el pare ho portarà a terme. Es a dir, aquesta variable no ha de ser global. • Visualització d'estat: S'ha de mostrar de forma combinada el temps total de partida i el temps restant de les "SUPER-PILOTES" a la línia de missatges inferior.

Implementació:

```

comptador_retard += retard;

if (temps_poder > 0) {
    temps_poder -= retard;
    if (temps_poder <= 0) {
        temps_poder = 0;
        waitS(id_sem);
        comp->poder_actiu = 0;
        signalS(id_sem);
    }
}

if (comptador_retard >= 1000) /* Ha passat 1 segon */
{
    segons++;
}

```

```

        if (segons == 60)
        {
            minuts++;
            segons = 0;
        }
        comptador_retard = 0;
    }

    if (temps_poder > 0)
        sprintf(strin, "Temps: %02d:%02d | Blocs: %d | Pilotes: %d | PODER: %.1fs", minuts, segons, comp->nblocs, comp->npilotes, temps_poder / 1000.0);
    else
        sprintf(strin, "Temps: %02d:%02d | Blocs: %d | Pilotes: %d", minuts, segons, comp->nblocs, comp->npilotes);

```

Pas 4.4: Comunicació de dades entre pilotes (Bústia)

Especificació:

S'implementa un mecanisme de cooperació on les pilotes que surten per la porteria influeixen en les que queden vives. • **Sacrifici de la pilota:** Just abans de finalitzar, la pilota envia la seva velocitat vertical a la bústia de missatges (sendM) per a cadascuna de les pilotes restants. • **Algorisme d'ajust:** En rebre la velocitat d'una pilota "sacrificada", el fil receptor incrementa o decrementa la velocitat de la seva pilota accelerant-la o alentint-la segons el valor rebut, respectant sempre els límits de velocitat establerts .

Implementació:

- Implementacio Sacrifici de la pilota en pilota4.c:

```

do
{
    autormsg.desti = 'N'; // N = Ningú
    autormsg.origen = 'F'; // F = Fill
    sendM(id_mis, &autormsg, sizeof(missatge));

    receiveM(id_mis, &missatge);
    if (missatge.origen == 'F' && missatge.desti == 'F'){
        vel_f = -vel_f + missatge.vel_f_pilota;
        vel_c = -vel_c + missatge.vel_c_pilota;
    }
}

```

Pas 4.5: Sincronització entre els fils d'un mateix procés.

Especificació:

Atès que cada procés pot disposar de varis fils d'execució, cal garantir la coherència

interna . • Mutex POSIX: S'utilitza un pthread_mutex_t sempre que sigui possible. •

Reflexió final: Aquesta fase il·lustra la convivència de semàfors IPC per a la sincronització global entre processos i mutexes de pthreads per a la sincronització interna de cada procés multifil

Implementació:

```
        if (possible != 0){
            for (int i = 0; i < paletes[pal].m_pal; i++){
                waitS(id_sem);
                win_escricar(paletes[pal].f_pal,
paletes[pal].c_pal + i, ' ', NO_INV);
                signalS(id_sem);
            }

            for (int i = 0; i < paletes[pal].m_pal; i++){
                waitS(id_sem);
                win_escricar(paletes[pal].f_pal + possible,
paletes[pal].c_pal + i, (char)(num_paleta + '0'), INVERS);
                signalS(id_sem);
            }
            pthread_mutex_lock(&mutex);
            paletes[pal].f_pal += possible;
            pthread_mutex_unlock(&mutex);
        }
    }

    if (paletes[pal].dirPaleta == 1) // Dreta
    {
        if ((paletes[pal].c_pal + paletes[pal].m_pal) < n_col -
1)
        {
            /* Esborrar l'extrem esquerre i pintar el nou extrem
dret */

            waitS(id_sem);
            win_escricar(paletes[pal].f_pal, paletes[pal].c_pal,
' ', NO_INV);

            signalS(id_sem);
            pthread_mutex_lock(&mutex);
            paletes[pal].c_pal++;
            pthread_mutex_unlock(&mutex);
            waitS(id_sem);
```



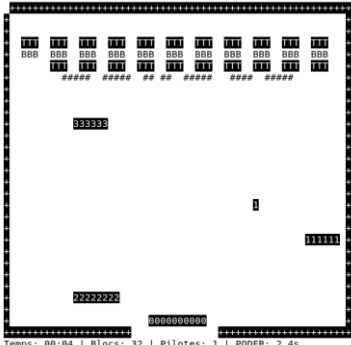
```

        win_escricar(paletes[pal].f_pal, paletes[pal].c_pal +
paletes[pal].m_pal - 1, (char)(num_paleta + '0'), INVERS);
        signalS(id_sem);
    }
    else
    {
        pthread_mutex_lock(&mutex);
        paletes[pal].dirPaleta = -1; // Canvia de direcció
        pthread_mutex_unlock(&mutex);
    }
}
else if (paletes[pal].dirPaleta == -1) // Esquerra
{
    if (paletes[pal].c_pal > 1)
    {
        /* Esborrar l'extrem dret i pintar el nou extrem
esquerre */

        waitS(id_sem);
        win_escricar(paletes[pal].f_pal, paletes[pal].c_pal +
paletes[pal].m_pal - 1, ' ', NO_INV);
        signalS(id_sem);
        pthread_mutex_lock(&mutex);
        paletes[pal].c_pal--;
        pthread_mutex_unlock(&mutex);
        waitS(id_sem);
        win_escricar(paletes[pal].f_pal, paletes[pal].c_pal,
(char)(num_paleta + '0'), INVERS);
        signalS(id_sem);
    }
    else
    {
        pthread_mutex_lock(&mutex);
        paletes[pal].dirPaleta = 1; // Canvia de direcció
        pthread_mutex_unlock(&mutex);
    }
}
}

```

Jocs de proves:

Prova realitzada	Resultat esperat	Resultat obtingut	Conclusio
make fase4	Mur4, pilota4, winsuport2, memori, semàfor, missatge compilats	<pre>milax@casa:~/Documents/GitHub/PracF50/mur\$ make fase4 gcc -Wall -g -c mur4.c -o mur4.o -lpthread gcc -Wall -g -c winsuport2.c -o winsuport2.o gcc -Wall -g -c memoria.c -o memoria.o gcc -Wall -g -c semafor.c -o semafor.o gcc -Wall -g -c missatge.c -o missatge.o gcc -Wall -g mur4.o winsuport2.o memoria.o semafor.o missatge.o -o mur4 gcc -Wall -g -c pilota4.c -o pilota4.o gcc -Wall -g pilota4.o winsuport2.o memoria.o semafor.o missatge.o -o pilota4</pre>	OK
./mur4 params_paleta.t xt 50	Blocs T i temps amb poder funcionen correctament	 <pre>Temps: 00:04 Blocs: 32 Pilotes: 1 PODER: 2.4s</pre>	OK